Week 1, Day 7 (Weekend): SOLID, Revision and Week 1 Review

Prepared for Upendra Kumar Sharma | 2.5–3 hours (light day) | Main topic: SOLID principles | Plus: notes revision and light DSA

Today's Schedule

- **0:00–1:00** Study SOLID with the code examples (section 1). Rewrite each "fixed" example yourself.
- **1:00–1:20** Practice explaining SOLID with an example from your own projects (section 2).
- **1:20–2:05** Revise Week 1: rapid-recall sheet and flashcards (section 3).
- **2:05–2:50** Light DSA: redo 4 problems and warm up with timed practice (section 4).
- **2:50–3:10** Week 1 review and Week 2 preparation (sections 5 and 6).

1. SOLID Principles

SOLID is five design guidelines that keep object-oriented code easy to change, test and extend. Interviewers ask for a definition, a bad example, and the fix. Know the "why" behind each, not just the names.

S: Single Responsibility Principle

A class should have only one reason to change. Give each class one job, so a change in one concern (such as email format) does not force edits in unrelated code (such as database logic).

```
// Bad: validation, persistence and notification in one class
class OrderService {
    void placeOrder(Order o) {
        if (o.items().isEmpty()) throw new IllegalArgumentException();
        // save to database ...
        // send confirmation email ...
    }
}

// Good: each class has one responsibility
class OrderValidator  { void validate(Order o) { /* rules */ } }
class OrderRepository { void save(Order o) { /* persistence */ } }
class OrderNotifier   { void confirm(Order o) { /* email */ } }
```

```
class OrderService {
    private final OrderValidator validator;
    private final OrderRepository repository;
    private final OrderNotifier notifier;
    // constructor injection omitted for brevity
    void placeOrder(Order o) {
        validator.validate(o);
        repository.save(o);
        notifier.confirm(o);
    }
}
```

O: Open/Closed Principle

Software entities should be open for extension but closed for modification. Add new behavior by adding new code, not by editing tested code. The usual tool is polymorphism (the Strategy pattern).

```
// Bad: every new customer type forces an edit to this method
double discount(String type, double amount) {
    if (type.equals("REGULAR")) return amount * 0.05;
    if (type.equals("PREMIUM")) return amount * 0.10;
    return 0;
}

// Good: add a new class instead of editing existing code
interface DiscountPolicy { double apply(double amount); }
class RegularDiscount implements DiscountPolicy {
    public double apply(double a) { return a * 0.05; }
}
class PremiumDiscount implements DiscountPolicy {
    public double apply(double a) { return a * 0.10; }
}
```

L: Liskov Substitution Principle

Objects of a subclass must be usable wherever the parent type is expected, without breaking correctness. A subclass must not strengthen preconditions, weaken postconditions, or throw unexpected exceptions.

```
// Bad: Penguin is a Bird, but fly() cannot work for it
class Bird { void fly() { /* ... */ } }
class Penguin extends Bird {
    @Override void fly() { throw new UnsupportedOperationException(); }
}

// Good: model the capability separately
class Bird { void eat() { /* ... */ } }
interface Flyable { void fly(); }
class Sparrow extends Bird implements Flyable { public void fly() { /* ... */
} }
class Penguin extends Bird { /* no fly() */ }
```

A classic variant is Square extending Rectangle. Setting width on a Square also changes its height, which surprises code written for Rectangles.

I: Interface Segregation Principle

Clients should not be forced to depend on methods they do not use. Prefer several small, focused interfaces over one large one.

```
// Bad: a basic printer is forced to implement scan() and fax()
interface Machine { void print(); void scan(); void fax(); }

// Good
interface Printer { void print(); }
interface Scanner { void scan(); }
class BasicPrinter implements Printer { public void print() { /* ... */ } }
class OfficeMachine implements Printer, Scanner {
    public void print() { /* ... */ }
    public void scan()  { /* ... */ }
}
```

D: Dependency Inversion Principle

High-level modules should not depend on low-level modules. Both should depend on abstractions. This is what Spring's dependency injection makes practical, and it makes code easy to test with mocks.

```
// Bad: tied to one concrete implementation
class NotificationService {
    private final SmtpEmailSender sender = new SmtpEmailSender();
}

// Good: depends on an abstraction, injected from outside
interface MessageSender { void send(String to, String body); }
class SmtpEmailSender implements MessageSender { /* ... */ }
class SmsSender implements MessageSender { /* ... */ }

@Service
class NotificationService {
    private final MessageSender sender;
    NotificationService(MessageSender sender) { this.sender = sender; }
    void notifyUser(String to, String msg) { sender.send(to, msg); }
}
```

Note: Dependency Inversion (the principle) is not the same as Dependency Injection (the technique) or Inversion of Control (the broader idea). Injection is one way to achieve inversion.

2. Explaining SOLID with Your Own Experience

Interviewers value real examples more than textbook ones. Prepare one example per principle from your own projects. Use these only where they match what you actually built:

- **Strategy (Open/Closed):** different workflows or validation rules per application type, such as New, Renewal, Amendment and Surrender in a licensing system, can be separate strategy classes instead of one big if/else.
- **Single Responsibility:** controller handles HTTP, service handles business rules, repository handles database access, and a separate component handles document or notification integration.
- **Dependency Inversion:** an interface for an external integration (DigiLocker, Alfresco, face recognition) with a mock implementation for tests.
- **Interface Segregation:** separate read-only and write interfaces for a document store, so a reporting module cannot modify documents.

Answer structure: name the principle, show what was going wrong or hard to change, what you did, and the benefit (easier testing, fewer regressions, faster feature addition).

3. Week 1 Rapid-Recall Sheet

Read each line, cover the answer, and try to say it from memory.

Collections and hashing

- HashMap: index = (n - 1) & hash. Default capacity 16, load factor 0.75. A bin converts to a tree at 8 nodes if capacity is at least 64.
- ConcurrentHashMap (Java 8+): CAS for empty bins, synchronized on the first node otherwise, no null keys or values.
- If equals() says equal, hashCode() must match. Never use mutable keys.
- Fail-fast iterators throw ConcurrentModificationException. Use Iterator.remove() or removeIf.
- Comparable defines natural ordering. Comparator defines external orderings. Use Integer.compare, not subtraction.

JVM

- Heap: young (Eden, S0, S1) and old. Metaspace replaced PermGen in Java 8 and lives in native memory.
- G1 is the default since Java 9. ZGC targets very low pauses. Parallel targets throughput.
- Leaks come from static collections, unbounded caches, unremoved listeners and ThreadLocals in pools.

Concurrency

- synchronized gives mutual exclusion and visibility. volatile gives visibility and ordering only, not atomicity.
- Double-checked locking needs a volatile field. wait() belongs in a while loop.
- ThreadPoolExecutor: core threads, then queue, then extra threads up to max, then reject.
- Deadlock needs mutual exclusion, hold and wait, no preemption and circular wait. Break it with a global lock order.
- ThreadLocal in pools: always remove() in finally.

Modern Java

- thenApply is map, thenCompose is flatMap. Pass your own executor for blocking I/O.
- Collectors.toMap throws on duplicate keys without a merge function.
- Parallel streams share the common pool, so use them only for large CPU-bound work.
- Records (16), sealed classes (17), switch expressions (14), text blocks (15), var (10).

Flashcards

- Review the 10 flashcards you wrote each day. Sort them into "know it", "shaky" and "don't know".
- Spend most of the time on the "don't know" pile, then reread the matching day guide section.

4. Light DSA Revision

The goal today is retention, not new material.

1. Redo **four** problems you found hardest this week, with a 25-minute timer each. Good candidates: Minimum Window Substring, Trapping Rain Water, Subarray Sum Equals K, 3Sum.
2. For each, say the pattern name out loud (sliding window, prefix sum with hashmap, two pointers, and so on) and its complexity before you start coding.
3. If time remains, solve one new easy array or string problem as a warm-up for Week 2.

Pattern cheat sheet from Week 1

- **Hash map lookup:** Two Sum, Group Anagrams.
- **Prefix sum plus hash map:** Subarray Sum Equals K.
- **Hash set to find sequence starts:** Longest Consecutive Sequence.
- **Two pointers on sorted data:** 3Sum, Valid Palindrome.
- **Two pointers moving the weaker side:** Container With Most Water, Trapping Rain Water.
- **Fixed window:** Maximum Average Subarray, Permutation in String.
- **Variable window with counts:** Longest Substring Without Repeating Characters, Minimum Window Substring.

5. SOLID and Design Interview Questions to Answer Aloud

1. Explain each SOLID principle with an example.
2. How does the Open/Closed Principle relate to the Strategy pattern?
3. Give an example of a Liskov Substitution violation and how to fix it.
4. What is the difference between Dependency Inversion, Dependency Injection and Inversion of Control?
5. Why does Spring prefer constructor injection over field injection?
6. How does following SOLID make code easier to test?
7. Can you over-apply SOLID? What does it look like when you do?

8. Show a place in your own code that violated Single Responsibility and how you improved it.

6. Week 1 Review and Week 2 Preparation

Week 1 review (write answers in your notes)

- What was my self-mock score on Day 6, and which topics scored 0 or 1?
- Which three DSA problems took me the longest, and what pattern did I miss?
- Did I stick to the daily schedule? What got in the way, and what will I change?
- Which topic do I want to revisit before the Week 3 mock?

Week 2 preview: Spring, JPA, SQL and Security

- Day 1: IoC and DI, bean lifecycle, scopes, circular dependencies. DSA: binary search.
- Day 2: AOP, proxies, auto-configuration, starters. DSA: stacks.
- Day 3: @Transactional propagation and isolation. DSA: intervals.
- Day 4: JPA and Hibernate, N+1, caching. DSA: linked lists.
- Day 5: SQL indexing, query plans, locking. DSA/SQL practice.
- Day 6: Spring Security, JWT, REST best practices.
- Day 7: revision and mock interview.

Before you start Week 2

- Have a small Spring Boot project open (even the one you plan to extend into the mini project) so you can try each concept in code as you study it.
- Block your daily study slot in your calendar for the next 7 days.

7. End-of-Day Checklist

- [ ] I can state each SOLID principle and give a bad and good code example.
- [ ] I have one real example from my own work for at least three principles.
- [ ] I reviewed my Week 1 flashcards and re-read sections for the "don't know" pile.
- [ ] I redid four hard DSA problems and named the pattern for each.
- [ ] I wrote my Week 1 review notes.

Tomorrow (Week 2, Day 1): IoC and DI, bean lifecycle, scopes and circular dependencies, plus binary search problems.